

Project Report

Bank Account Management System

Student : **Menda Kalyani**

Registration no: **25BAI11272**

Branch: **CSE- AI ML**

Slot: **B11+B12**

Professor: **Dr. Dheresh Soni**

VITYARTHI : Java Programming Project

1. Introduction

Banking is a part of everyday life, and people regularly perform activities such as depositing money, withdrawing money, and checking their account balance. Although these operations are simple for a user, they require proper handling of account information and calculations.

MyBank – Bank Account Management System is a simple Java-based console application developed to perform some of these basic banking operations. The program provides a menu through which the user can create an account, view account details, deposit money, withdraw money, check the balance, and exit the application.

I developed this project mainly to understand how Java programming concepts can be used to solve a practical problem. It also helped me practice concepts such as classes, objects, constructors, methods, conditional statements, loops, and user input.

2. Abstract

MyBank is a basic Bank Account Management System developed using Java. The application provides a simple way to perform common operations related to a bank account through a console interface.

When the program starts, the user is given different options. The user can create an account by entering an account number, account holder name, and initial deposit. After creating the account, the user can deposit or withdraw money and view the current account information.

The program also includes basic checks. For example, it does not allow a negative or zero amount to be deposited or withdrawn, and it prevents a withdrawal when the available balance is not sufficient.

The project is mainly intended for learning purposes and demonstrates how basic Java and Object-Oriented Programming concepts can be applied to a real-world example.

3. Problem Statement

Managing account information and performing simple transactions manually can become inconvenient. There is also a possibility of making mistakes while calculating or updating the balance.

The purpose of this project is to create a simple program that can handle basic bank account operations in an organized manner. The system allows the user to create an account and perform deposits, withdrawals, and balance-related operations from a single console application.

4. Objectives

The main objectives of this project are to:

- Develop a simple banking application using Java.
 - Understand how real-world banking operations can be represented through programming.
 - Implement account creation, deposit, withdrawal, and balance checking.
 - Learn how classes and objects can be used in a practical application.
 - Understand the use of constructors and methods.
 - Practice taking input from users using the Scanner class.
 - Apply conditional statements and loops.
 - Add basic validation to prevent incorrect transactions.
 - Improve logical thinking and programming skills.
-

5. Scope of the Project

The current version of MyBank focuses on basic management of a single bank account.

The user can create an account by providing an account number, account holder name, and initial deposit. Once the account has been created, the user can display the account information, deposit money, withdraw money, and check the available balance.

The project is intentionally kept simple because its main purpose is to demonstrate Java programming concepts. It does not try to provide all the features or security requirements of a real banking application.

6. Target Users

MyBank is mainly suitable for students and beginners who are learning Java programming.

For students, the project provides an example of how programming concepts can be applied to a real-life situation. Beginners can also use the project to understand how classes, objects, methods, loops, conditions, and user input work together in a complete application.

The project can also be used as an academic project to demonstrate basic Object-Oriented Programming skills.

7. Technologies Used

The project is developed using **Java** as the main programming language.

Visual Studio Code is used as the development environment for writing and running the program.

The project uses the Java Scanner class for taking input from the user. Basic Object-Oriented Programming concepts are used to organize the account-related information and operations.

Git and GitHub can be used to maintain and share the source code of the project.

8. Features of the Application

Account Creation

The user can create a bank account by entering an account number, account holder name, and initial deposit.

After receiving these details, the program creates an object of the `BankAccount` class and stores the information.

Display Account

The user can view the details of the currently created account. The program displays the account number, account holder name, and current balance.

If an account has not been created, the program informs the user that no account exists.

Deposit Money

The deposit option allows the user to add money to the account.

Before depositing the amount, the program checks whether the amount is greater than zero. If the amount is valid, it is added to the existing balance and a confirmation message is displayed.

Withdraw Money

The withdrawal option allows the user to take money from the account.

The program first checks whether the entered amount is valid. It then checks whether enough money is available in the account.

If the withdrawal amount is greater than the current balance, the program displays an "Insufficient balance" message instead of completing the transaction.

Check Balance

The user can select the balance option to view the current account balance.

In the current implementation, the `displayAccount()` method is used to show the account information along with the balance.

Exit

The user can select the exit option when they are finished using the application. The program displays a thank-you message and terminates.

9. Working of the Application

The program starts by displaying a welcome message and a list of available operations.

The user enters a choice from the menu. The program then uses a `switch` statement to determine which operation should be performed.

If the user chooses to create an account, the program asks for the account number, account holder name, and initial deposit. These values are then passed to the `BankAccount` constructor.

For other operations, the program first checks whether an account has already been created. If there is no account, the user is asked to create one first.

The program continues displaying the menu after every operation. This is handled using a `do-while` loop. The loop continues until the user selects option 6, which is the exit option.

10. Implementation

The complete application is currently written in a single Java file called **MyBank.java**.

The program contains two classes: `BankAccount` and `MyBank`.

The BankAccount class is responsible for storing account information and performing account-related operations. It contains three private variables: accountNumber, accountHolder, and balance.

The constructor is used to initialize these values when a new account is created.

The class also contains methods such as deposit(), withdraw(), and displayAccount().

The MyBank class contains the main() method. It controls the overall application and handles user interaction. It creates the Scanner object, displays the menu, accepts the user's choice, and calls the appropriate operation.

11. Object-Oriented Programming Concepts Used

One of the main reasons for developing this project was to get practical experience with Object-Oriented Programming.

Classes and Objects

The BankAccount class represents a bank account. When the user creates an account, an object of this class is created.

```
account = new BankAccount(  
    accNo,  
    name,  
    initialDeposit  
);
```

This object stores the information of the account while the program is running.

Constructor

The BankAccount class contains a constructor that receives the account number, account holder name, and initial balance.

The constructor initializes the object when it is created.

Encapsulation

The account information is declared using the private access modifier.

```
private int accountNumber;  
private String accountHolder;  
private double balance;
```

This prevents these variables from being directly accessed from outside the class.

Methods

Different operations are implemented using methods. For example, the `deposit()` method handles deposits, while the `withdraw()` method handles withdrawals.

This makes the code more organized because each method has a specific responsibility.

12. Input Handling

The program uses the `Scanner` class to take information from the user.

```
Scanner sc = new Scanner(System.in);
```

Different methods of `Scanner` are used depending on the type of information being entered.

`nextInt()` is used for integer values such as the account number and menu choice.

`nextDouble()` is used for amounts such as the initial deposit, deposit amount, and withdrawal amount.

`nextLine()` is used to accept the account holder's name.

The program also uses `sc.nextLine()` after taking the account number to properly handle the newline before reading the name.

13. Validation

Basic validation has been included in the application to avoid some incorrect operations.

For deposits, the program checks whether the amount is greater than zero. If it is not, the program displays "Invalid amount."

The same check is performed for withdrawals.

For withdrawals, the program also compares the requested amount with the current balance. If the requested amount is greater than the balance, the withdrawal is not completed and the user receives an "Insufficient balance" message.

The program also checks whether an account has been created before allowing operations such as depositing or withdrawing money.

14. Control Structures Used

Several basic Java control structures are used in the project.

The do-while loop keeps the application running and repeatedly displays the banking menu until the user chooses to exit.

The switch statement handles the different menu choices. Each case corresponds to a particular operation.

if-else statements are used for checking conditions such as whether an account exists, whether an amount is valid, and whether sufficient balance is available.

These control structures are important parts of the program because they determine how the application responds to different user inputs.

15. Testing

The application was designed to handle both normal and incorrect inputs.

For example, after creating an account with an initial balance, depositing money should increase the balance. Similarly, a successful withdrawal should reduce the balance.

If a user enters zero or a negative value while depositing or withdrawing, the program displays an invalid amount message.

Another important test is trying to withdraw more money than the account contains. In this situation, the program displays an insufficient balance message and does not reduce the balance.

The program also handles situations where the user tries to perform an operation before creating an account by displaying a message asking them to create an account first.

16. Advantages

MyBank is simple and easy to understand, which makes it suitable for someone who is learning Java.

The project demonstrates several important programming concepts in one application instead of using isolated examples. It also provides a practical understanding of how data can be stored and modified using objects and methods.

Another advantage is that the application does not require a database or any complicated setup. It can be compiled and executed directly using Java.

The project can also be expanded later if additional programming concepts need to be introduced.

17. Limitations

The current version of MyBank has some limitations.

The program manages only one account during a single execution. It does not currently support multiple customers or multiple accounts.

The account information is stored only while the program is running. Once the program is closed, the information is lost because there is no file or database for permanent storage.

The application also uses a console interface, so it does not have a graphical user interface.

There is no login, PIN, password protection, or advanced security mechanism. The project is therefore meant for educational purposes and should not be considered a real banking system.

18. Future Enhancements

There are several features that could be added to make the project more complete.

The application could be modified to support multiple accounts and customers. Each customer could have a separate account number and balance.

A transaction history could also be added so that users can see their previous deposits and withdrawals.

A login system using an account number and PIN could provide basic authentication.

Another useful addition would be a money transfer feature that allows one account to transfer money to another account.

The project could also use file handling or a database such as MySQL to permanently store account information.

In the future, the console interface could be replaced with a graphical user interface using Java Swing or JavaFX.

19. Learning Outcomes

Working on MyBank provided practical experience with basic Java programming.

The project helped in understanding how classes and objects can be used to represent real-world entities. It also provided practice with constructors, methods, private variables, conditional statements, loops, switch statements, and user input.

Another important learning outcome was understanding how different programming concepts can be combined to create a complete working application rather than writing individual pieces of code separately.

The project also introduced the use of Git and GitHub for maintaining and sharing source code.

20. Project Structure

At present, the project is intentionally kept simple and contains the main source code in one Java file.

MyBank :

- MyBank.java
- README.md
- Statement.md

The MyBank.java file contains both the BankAccount class and the MyBank class.

As the project grows, these classes could be separated into different files to make the code easier to maintain.

21. Conclusion

MyBank is a simple Bank Account Management System developed using Java to understand the practical use of basic programming and Object-Oriented Programming concepts.

The application allows the user to create an account, view account details, deposit money, withdraw money, and check the account balance. It also includes basic validation for invalid amounts, insufficient balance, and operations performed before creating an account.

Although the current version is simple and has limitations, it successfully demonstrates how a real-world idea can be converted into a working Java application.

The project also provides a good base for future improvements such as multiple accounts, transaction history, authentication, database connectivity, money transfers, and a graphical interface.